J Heuristics (2008) 14: 271–283
DOI 10.1007/s10732-007-9038-y

A large neighborhood search heuristic for the longest common subsequence problem

Todd Easton · Abhilash Singireddy

Received: 22 February 2005 / Revised: 9 October 2006 / Accepted: 7 December 2006 /
Published online: 18 August 2007

Abstract Given a set $S = \{S^1, \ldots, S^k\}$ of finite strings, the k-Longest Common Subsequence Problem (k-LCSP) seeks a string L^* of maximum length such that L^* is a subsequence of each S^i for $i = 1, \ldots, k$. This paper presents a large neighborhood search technique that provides quality solutions to large k-LCSP instances. This heuristic runs in linear time in both the length of the sequences and the number of sequences. Some computational results are provided.

Keywords Longest common subsequence · Dynamic programming · Large neighborhood search · Heuristic

1 Introduction

Given a finite alphabet set Σ, a *string*, also known as a sequence, is an ordered set of symbols drawn from Σ (repeats are allowed). By deleting letters from a string, a subsequence can be created. Given finite strings $S^1, \ldots, S^k$, the *k-Longest Common Subsequence* problem (k-LCSP) seeks a string L^* of maximum length such that L^* is a subsequence of each S^i for $i = 1, \ldots, k$. The string L^* is referred to as a *longest common subsequence* (LCS) and $|L^*|$ equals the *length of a longest common subsequence* (LLCS). Without loss of generality, we may assume that $|S^i| = n$ for all $i = 1, \ldots, k$. If not, $n - |S^i|$ dummy letters could be added to the end of sequence S^i where n is the $\max_{i \in \{1,\ldots,k\}} |S^i|$.

T. Easton (✉) · A. Singireddy
School of Industrial and Manufacturing Systems Engineering, Kansas State University,
237 Durland Hall, Manhattan, KS 66506, USA
e-mail: teaston@ksu.edu

A. Singireddy
e-mail: singireddy@gmail.com

Numerous computer science applications of k-LCSP have been cited in the literature. These applications include file comparison (Hunt and McIlroy 1975; Hunt and Szymanski 1977; Aho et al. 1983), data compression (Wagner 1973; Storer 1988), syntactic pattern recognition (Lu and Fu 1978), artificial intelligence (Hayes 1989; Jiang and Li 1995), text editing (Sankoff and Kruskal 1983), and query optimization in database systems (Sellis 1988).

In addition, Dayhoff (1969), Smith and Waterman (1981), Sankoff and Kruskal (1983), Bafna et al. (1995) and Jiang et al. (2002) also studied k-LCSP with respect to the homology of macromolecules. Furthermore, Banerjee and Ghosh (2001) categorized visitors to a website based on their common website interactions and Brisk et al. (2004) noted k-LCSP's importance to produce smaller circuits in field programmable gate arrays.

For arbitrary k, k-LCSP is *NP*-complete (Maier 1978); however, if k is fixed, then k-LCSP can be solved in polynomial time by dynamic programming (Larson 1968). Most of k-LCSP's research has focused on instances with $k = 2$ and 3. However, Gallant et al. (1980), Itoga (1981), Hsu and Du (1984), Irving and Fraser (1992), Hakata and Imai (1992) and Singireddy (2003) have all focused on providing optimal solutions to instances with $k \geq 4$.

Very little work has been done on nonoptimal algorithms or heuristics for k-LCSP. In fact in 1994, Chin and Poon (1994) claim that there had been no heuristic work done on k-LCSP. These individuals then proceed to provide a heuristic which is similar to a long run algorithm simultaneously developed by Jiang and Li (1995). Bonizzoni et al. (1998) created an expansion algorithm that mixes ideas from long run and dynamic programming. Other work on heuristics for k-LCSP can be found in Guenoche and Vitte (1995), Bergroth et al. (1998) and Brisk et al. (2004).

The remainder of the paper is organized as follows. Section 2 explains large neighborhoods of k-LCSP and surveys methods that can determine the optimal common sequence in such neighborhoods. Section 3 describes the new heuristic, which is called the time horizon specialized branching heuristic (THSB). The paper concludes with computational results in Sect. 4 and some directions for future research in Sect. 5.

2 Large neighborhoods of k-LCSP

The core of the heuristic presented here deals with optimally solving large neighborhoods (Ahuja et al. 2002) of k-LCSP. Here we provide a general framework for these neighborhoods and Sect. 3 describes the particular neighborhoods used in THSB.

Given a set of k strings, a substring of each of the k strings, and a common subsequence L, the concept behind the neighborhood is to remove each substring from its string. By removing these substrings L will be reduced to two disjoint subsequences, L^1 and L^3. A common subsequence, L', is in the neighborhood of L if and only if L' can be represented as (L^1, L^2, L^3) where L^2 is any common subsequence of the removed substrings.

Formally, given sequences $S^1, \ldots, S^k$ with $S^i = (s_1^i, s_2^i, s_3^i, \ldots, s_n^i)$ for all $i = 1, \ldots, k$ and $L = (\sigma_1, \sigma_2, \ldots, \sigma_{|L|})$ a common subsequence. Let $j_1, j_1', j_2, j_2', \ldots, j_k, j_k'$ be any integers such that $1 \leq j_i \leq j_i' \leq n$ for all $i = 1, \ldots, k$. Define L^1 to

be $(\sigma_1, \sigma_2, \ldots, \sigma_q)$ such that either $q = |L|$ or σ_q occurs in each sequence before each $s^i_{j_i}$ and σ_{q+1} occurs in at least one sequence at or after $s^i_{j_i}$. Similarly, let L^3 be $(\sigma_r, \sigma_{r+1}, \ldots, \sigma_{|L|})$ such that either $r = 1$ or σ_r occurs in each sequence after each $s^i_{j'_i}$ and σ_{r-1} occurs in at least one sequence before or at $s^i_{j'_i}$. If no such q or r exist, then the appropriate L^1 or L^3 are set to the empty set. Let L^2 be any common subsequence of the k-LCSP instance generated from $S'^1 = (s^1_{j_1}, s^1_{j_1+1}, \ldots, s^1_{j'_1}), S'^2 = (s^2_{j_2}, s^2_{j_2+1}, \ldots, s^2_{j'_2}), \ldots, S'^k = (s^k_{j_k}, s^k_{j_k+1}, \ldots, s^k_{j'_k})$. Now define the $j_1, j'_1, j_2, j'_2, \ldots, j_k, j'_k$ neighborhood of L, denoted as $N^L(j_1, j'_1, j_2, j'_2, \ldots, j_k, j'_k)$, to be any sequence of the form (L^1, L^2, L^3). Clearly, every sequence in $N^L(j_1, j'_1, j_2, j'_2, \ldots, j_k, j'_k)$ is a common subsequence of $S^1, \ldots, S^k$.

As is typically the case for large neighborhoods, the presented heuristic seeks the best common subsequence in $N^L(j_1, j'_1, j_2, j'_2, \ldots, j_k, j'_k)$. Clearly, the best such sequence is (L^1, L^{2^*}, L^3) where L^{2^*} is an optimal solution to the k-LCSP instance consisting of $S'^1 = (s^1_{j_1}, s^1_{j_1+1}, \ldots, s^1_{j'_1}), S'^2 = (s^2_{j_2}, s^2_{j_2+1}, \ldots, s^2_{j'_2}), \ldots, S'^k = (s^k_{j_k}, s^k_{j_k+1}, \ldots, s^k_{j'_k})$. Thus, quickly solving small k-LCSP instances (large k and small n) is critical to the heuristic's success and so the attention turns toward techniques that optimally solve k-LCSP instances.

2.1 Solving k-LCSP

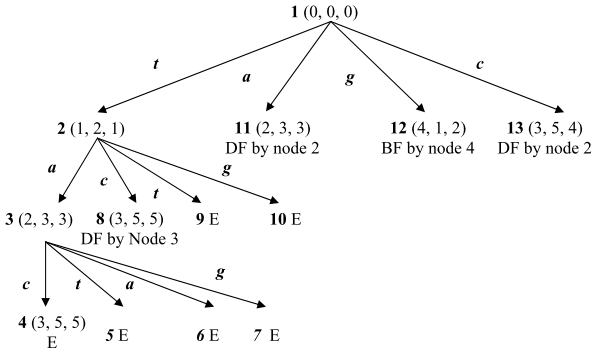
To the best of the authors' knowledge, there exists only three main categories of algorithms that optimally solve k-LCSP. These are dynamic programming, enumeration and integer programming. Only a brief description of each of these techniques with some relevant references is provided here.

Dynamic programming (DP) is by far the most well known and researched technique that solves k-LCSP. Standard dynamic programming requires $O(n^k)$ time and storage to solve k-LCSP. A more complete description can be found in Larson (1968). Hirschberg (1975), Eppstein et al. (1992), Irving and Fraser (1992), Hakata and Imai (1992) and many others have all suggested numerous improvements to dynamic programming, and these improvements typically reduce either the storage or run time from $O(n^k)$ to $O(n^{k-1})$. Bergroth et al. (2000) provide a survey of many of these advancements.

An effective enumeration algorithm was first proposed by Hsu and Du (1984). They built a branching tree where each node in this tree has $|\Sigma|$ children and each child corresponds to a different letter as the next letter in the common subsequence. Then they found a longest rooted path, which corresponded to the LCS.

Recently Singireddy (2003) improved this enumeration technique by including two new methods to prune/fathom the branching tree. A node can be fathomed if proceeding further down its ancestors cannot produce a better solution than the best current solution. This type of fathoming is based upon concepts from branch and bound (Land and Doig 1960). A node can also be fathomed if the current best solution has a better start than the current node. This type of fathoming is taken from dynamic programming concepts. This enumeration/branching technique with these two fathoming concepts is called specialized branching (SB) and requires $O(k|\Sigma|^{|L^*|})$ effort where L^* is the optimal solution to the k-LCSP instance.

Fig. 1 SB's branching tree



SB's branching tree for the sequences *tacga*, *gtaac* and *tgacc* is in Fig. 1. The procedure branches depth first left and orders the branches from left to right in terms of the smallest maximum position of the branched upon letter. An E in the tree indicates an end of the sequence occurred. Both nodes 11 and 13 are fathomed since node 2 has at least as good of a start. Also observe that branching further down node 12 can lead to at most one additional letter and so node 12 can be fathomed by node 4. Thus, the optimal solution occurs at node 4 with $L^* = tac$.

An integer programming model was developed in (Singireddy 2003). This method models the k-LCSP as a multilayer graph and then seeks the maximum number of non-crossing paths. Facets for this problem were obtained and implemented in a branch and cut algorithm to solve k-LCSP. For fixed k, this technique was shown to be polynomial, but requires at least $O(n^k)$ effort. This algorithm will be referred to as BC.

2.2 Finding the optimal solution to a neighborhood

Table 1 shows a computational study between DP, SB and BC and is intended to determine which method should be used to obtain L^{2^*} (find the best common subsequence in $N^L(j_1, j_1', j_2, j_2', \ldots, j_k, j_k')$). Each problem in Table 1 was randomly generated with $|\Sigma| = 4$. Each row in Table 1 represents the average of 5 instances of the specified size. An N/A in the table represents that none of the 5 instances were solved within the time limit of 18,000 seconds (5 hrs). If there is a superscript above a number, then this superscript is the number of instances that were solved within the time frame. If there is no superscript, then each of the 5 instances was solved. All computational results presented in this paper were performed on a 1.5 GHz Pentium IV PC with 512 MB of RAM.

Clearly, SB computationally dominates BC. Furthermore SB dominates DP for large k. When applying DP, only the standard version was implemented. Including some of its bells and whistles (Hakata and Imai 1992; Hirschberg 1977; Irving and Fraser 1992) would improve the computational time; however, DP would still be substantially inferior to SB for any $k \geq 6$. Table 1 provides ample evidence that SB is the best method to solve k-LCSP instances with small n and large k. Ta-

Table 1 A comparison of DP, SB and BC

No. of Strings	String Length	Avg. LLCS	Min–Max LLCS	Avg. Time (DP)	Avg. Time (SB)	Min–Max Time (SB)	Avg. Time (BC)	Min–Max Time (BC)
2	25	14	12–15	0	0	0–0	4	1–8
	50	29	27–31	0	0	0–0	11066[3]	1871–17182
	75	45	44–47	0	44	2–108	N/A	N/A
	100	62	60–63	0	407	25–906	N/A	N/A
	1000	648	643–656	0	N/A	N/A	N/A	N/A
	10000	6526	6515–6541	17	N/A	N/A	N/A	N/A
4	25	9	7–10	0	0	0–0	1808[4]	812–2920
	50	21	20–23	2	1	0–3	N/A	N/A
	75	33	30–35	9	931	76–1862	N/A	N/A
	100	45	44–46	45	N/A	N/A	N/A	N/A
	1000	N/A	N/A	N/A	N/A	N/A	N/A	N/A
6	25	8	7–8	N/A	0	0–0	15647[2]	13589–17705
	50	17	16–18	N/A	3	1–6	N/A	N/A
	75	29	27–29	N/A	2936	1141–5793	N/A	N/A
	100	N/A	N/A	N/A	N/A	N/A	N/A	N/A
10	25	7	6–7	N/A	0	0–0	N/A	N/A
	50	15	14–16	N/A	2	1–4	N/A	N/A
	75	24	23–26	N/A	3731	1610–6982	N/A	N/A
	100	N/A	N/A	N/A	N/A	N/A	N/A	N/A
50	25	3	3–4	N/A	0	0–0	N/A	N/A
	50	10	10–11	N/A	1	0–1	N/A	N/A
	75	17	17–18	N/A	505	259–939	N/A	N/A
	100	N/A	N/A	N/A	N/A	N/A	N/A	N/A
100	25	3	2–3	N/A	0	0–0	N/A	N/A
	50	9	8–9	N/A	0	0–1	N/A	N/A
	75	15	15 –16	N/A	320	206–460	N/A	N/A
	100	N/A	N/A	N/A	N/A	N/A	N/A	N/A
1000	25	1	1–1	N/A	0	0–0	N/A	N/A
	50	6	5–6	N/A	1	1–1	N/A	N/A
	75	11	11–11	N/A	372	310–547	N/A	N/A
	100	N/A	N/A	N/A	N/A	N/A	N/A	N/A

ble 1 also demonstrates that for random k-LCSP instances with $|\Sigma| = 4$, SB is truly only effective when n is less than 75. Therefore, SB is used to provide optimal solutions to the large neighborhoods presented in Sect. 2 (i.e. solve for L^{2^*}). Additional computational studies on SB can be found in (Singireddy 2003).

3 The time horizon specialized branching heuristic

The aforementioned neighborhoods create a framework for numerous distinct heuristics. THSB uses a p neighborhood. An $N^L(j_1, j_1', j_2, j_2', \ldots, j_k, j_k')$ neighborhood is called a p neighborhood, denoted by $N_p^L(j_1, j_2, \ldots, j_k)$, if $j_1' = j_2' = \cdots = j_k' = \min\{\max\{j_1, j_2, \ldots, j_k\} + p, n\}$. Thus a p neighborhood will have the exact same ending location for each of the removed substrings.

To provide fast and accurate solutions, THSB implements these p neighborhoods using a common practice from infinite time horizon problems (cf. Winston 2004). This common practice solves the problem for some limited number of periods, say the first p periods, but only uses the solution for the first r periods, where $r < p$. In this fashion, the implemented solutions can avoid the nonoptimal solutions that are caused by artificially limiting the number of periods. As expected, these nonoptimal solutions typically occur near the truncated horizon. Since specialized branching is used to solve for L^{2^*}, the heuristic is called the Time Horizon Specialized Branching Heuristic (THSB).

To extend this time horizon strategy to a k-LCSP instance, let each letter represent a time period. Thus, THSB works sequentially starting with the first letters. Any letter (time horizon) further than $\max\{j_1, j_2, \ldots, j_k\} + p$ is ignored ($L^3 = \varnothing$). Second, THSB only appends the first $q|L^{2^*}|$ letters of L^{2^*} where $q \in [0, 1]$ (the entire optimal solution is not followed). In other words, SB is used to find an optimal solution (L^{2^*}) to the k-LCSP problem created from the first p letters of each sequence and only the first $q|L^{2^*}|$ letters are used in the reported common subsequence. The process continues to repeat by creating a new k-LCSP instance starting with the letter in each sequence after the last letter added to the CS and ending with the $p + \rho$ letter in each sequence where ρ is equal to the maximum location of the last letter added to the CS. Again SB solves this instance and only $q|L^{2^*}|$ letters are appended to the CS. Formally,

3.1 Time horizon specialized branching heuristic (THSB)

Initialization: Set a marker m_i to 1 for $i = 1, \ldots, k$. Set the horizon size, p, to some integer and set $q \in [0, 1]$ to be the portion of the accepted solution. Set L to the empty set.

Main Step: While $m_i + p \leq |S^i|$ for all $i = 1, \ldots, k$, generate a k-LCSP instance by taking the m_i^{th} letter to the $(m + p)^{th}$ letter of each S^i where $m = \max\{m_1, m_2, \ldots, m_k\}$, i.e. create a $N_p^L(m_1, m_2, \ldots, m_k)$. Use SB to solve this instance, which produces an L^{2^*} (the letters of a longest common subsequence of the generated k-LCSP). If $q|L^{2^*}|$ rounded to the nearest integer is 0, then set each m_i to $m_i + 1$ for $i = 1, \ldots, k$. If not, append the first $q|L^{2^*}|$ (rounded to the nearest integer) letters of L^{2^*} to the end of L and set m_i to the location in S^i after the last letter appended to L for each $i = 1, \ldots, k$.

Termination Step: Generate a final k-LCSP instance by taking the m_i^{th} letter to the $|S^i|^{th}$ letter of each S^i for $i = 1, \ldots, k$ (create a $N^L(m_1, |S^1|, m_2, |S^2|, \ldots, m_k, |S^k|)$). Solve this k-LCSP using SB. To L, append the solution from this k-LCSP and then report L as the candidate solution for the k-LCSP instance.

The following example illustrates THSB. Let the input strings be *agctaatc*, *gtacgatc*, and *tatccatc* and set $p = 4$ and $q = .5$. THSB would begin by finding an LCS to *agct*, *gtac*, *tcac*, which is *ac*. Since $q = .5$, only a is kept, observe that the maximum location of this a is 3 and so the next problem solves until the seventh letter, i.e. find the LCS of *gctaat, cgat, tccat*. An optimal solution to this problem is *cat* and so we keep the first 1.5 letters, since we round up the first two letters are kept. So the current common subsequence is *aca*. The final problem solves *atc*, *tc* and *tc*. The optimal solution is *tc* and so the reported common subsequence is *acatc*, which is an optimal solution.

In implementing THSB, SB can be applied at most $n - p + 1$ instances. Each of SB's solutions requires $O(k|\Sigma|^{|L2^*|})$. Since $|L^{2^*}|$ is less than or equal to p, each SB instance solves in less than $O(k|\Sigma|^p)$ time. Consequently, THSB runs in $O(nk|\Sigma|^p)$ time. Therefore, the theoretical run time of THSB is linear in the number of sequences and linear in the length of these sequences. However, THSB is exponential in p. Fortunately, THSB is computationally fast for reasonable values of p.

4 Computational results

This computational study shows that THSB is a heuristic that provides excellent common subsequences in a reasonable amount of time. The problems selected for this computational study are randomly generated and are so immense and DP, SB and BC could never optimally solve these instances.

The instances in both Tables 2 and 3 are identical and have $|\Sigma| = 4$ with the probability of choosing a specific letter as $1/|\Sigma|$. Table 3 provides the size of these instances and each row reports the average value of 50 random instances of the specified size.

Table 2 is condensed and reports the average effect of both q and p on THSB's solution quality and time for all 900 instances. As expected, larger values of p enhance the solution quality, but require more time. Also as expected, smaller values of q tend to improve the solution quality, but require more time. From Table 2, we conclude that choosing q between .5 and .75 provides good solutions without too much additional time. Consequently, the remainder of the computational studies has q set to .66.

Table 2 The effect of p and q on THSB's solution quality and time

	$q = .25$		$q = .33$		$q = .5$		$q = .66$		$q = .75$		$q = 1.0$	
	Avg LCS	Avg Time	Avg LCS	Avg Time	Avg LCS	Avg Time	Avg LCS	Avg Time	Avg LCS	Avg Time	Avg LCS	Avg Time
$p = 20$	160.68	1.30	160.68	1.16	159.87	0.54	159.67	0.45	159.35	0.38	157.66	0.22
$p = 30$	162.25	10.14	162.23	8.51	162.35	4.73	162.10	3.78	161.64	2.89	160.40	1.31
$p = 40$	163.69	140.09	164.03	117.12	164.28	66.79	164.22	50.96	164.06	37.38	162.14	12.94
AVG	162.21	50.51	162.31	42.26	162.17	24.02	162.00	18.39	161.68	13.55	160.07	4.82

Table 3 The performance of THSB and other heuristics on large k-LCSP instances with $|\Sigma| = 4$

Num of Seq.	Length of Seq.	G and V		DP		LR		EA		THSB					
										$p = 20, q = .66$		$p = 30, q = .66$		$p = 40, q = .66$	
		Avg CS	Avg Time (s)	Avg CS	Avg Time (s)	Avg CS	Avg Time (s)	Avg CS	Avg Time (s)	Avg CS	Avg Time (s)	Avg CS	Avg Time (s)	Avg CS	Avg Time (s)
10	250	76.24	0.04	58.4	0.5	56.3	0.01	68.0	0.2	79.5	0.1	80.9	1.2	82.8	16.7
10	500	157.32	0.14	122.4	1.2	116.5	0.02	139.9	0.9	161.5	0.2	165.2	2.8	168.2	41.4
10	1000	322.22	0.19	250.4	3.3	237.3	0.02	281.8	3.6	324.4	0.5	332.7	5.9	338.7	87.3
50	250	59.3	0.18	19.9	4.5	50.9	0.02	54.5	19.6	60.8	0.1	61.8	1.1	62.6	12.3
50	500	124.32	0.2	45.9	14.1	108.6	0.02	116.6	83.7	127.0	0.3	128.3	2.6	129.7	33.7
50	1000	257.76	0.34	96.0	49.9	227.0	0.04	241.7	321.2	259.5	0.7	261.7	5.6	264.7	78.9
100	250	55.5	0.22	11.4	14.2	48.8	0.04	50.9	151.9	57.0	0.2	57.7	1.5	57.9	18.8
100	500	117.82	0.34	29.1	51.3	105.6	0.04	110.5	649.2	119.8	0.5	121.1	3.9	122.3	48.6
100	1000	244.3	0.58	65.0	182.7	222.2	0.08	233.9	2534.1	247.3	1.2	249.7	9.4	251.0	110.7
Average		157.2	0.25	77.6	35.7	130.4	0.03	144.2	418.3	159.7	0.4	162.1	3.8	164.2	49.8

Table 3 shows that THSB is linear in n and k. An interesting observation is that the problems with 50 sequences require about the same time as the problems with 10 sequences, which is better than the theoretical linear performance in k. This computational anomaly is due to the fact that an increase in k should decrease the size of the longest common subsequence. Thus, $|L^{2^*}|$ is expected to decrease and so each of SB's branching trees is not as deep (requires less time), which cancels the increased computational time due to the increase in k.

Table 3 also compares THSB to four existing k-LCSP heuristics, Guenoche and Vitte, dynamic programming heuristic, long run and expansion algorithm. Each of these heuristics reports a common subsequence.

Guenoche and Vitte's (1995) (G and V) developed a linear time heuristic for k-LCSP. Their premise is to develop an objective function and select the next letter in the common sequence which minimizes the objective function. This method is run both forward and backward, and then the two solutions are merged. In implementing Guenoche and Vitte's method, we used the four different objective functions described in Guenoche (2004) and the best LCS from these 4 methods was recorded along with the time to evaluate all four methods.

The dynamic programming heuristic (DPH) as described in Guenoche (2004) is related to a phylogenic tree (Thompson et al. 1994). Briefly, DPH solves all pairs of dynamic programs (There are $\binom{k}{2}$ such DPs to solve.) and then creates a phylogenic tree based upon the length of the common subsequence between any two sequences. The main step of the heuristic replaces two strings with their longest common subsequence, found by dynamic programming. The phylogenetic tree provides an ordering for this replacement. The heuristic terminates with a single string, which is a common subsequence of all of the strings.

The long run algorithm (LR), developed by (Jiang and Li 1995), is also used to assess the accuracy of THSB. LR examines the sequences and reports a CS that is a longest common subsequence consisting of a single letter. LR is shown to always be within a factor of $|\Sigma|$ of the optimal solution.

The expansion algorithm (EA), introduced in Bonizzoni et al. (1998), guarantees to perform at least as well as LR. EA seeks out a set of two letters and then uses dynamic programming to expand these two letters to a CS that has only these letters.

Table 3 demonstrates THSB's dominance over the other 4 methods. THSB with $p = 40$ and $q = .66$ created solutions that are on the average 4.5% better than Guenoche and Vitte's method, 111.6% better than DPH, 25.9% better than LR and 13.9% better than EA. Furthermore, THSB with $p = 40$ achieved the best CS on all 900 of these instances.

It should be noted that THSB with $p = 40$ averaged about a minute to solve each of these problems and about 2 minutes to solve the largest problems. In contrast, Guenoche and Vitte required less than a second to provide a solution. However, if time is a critical factor, then THSB with $p = 30$ averaged less than 4 seconds to solve any single problem. And THSB provided solutions that are on average 3.1% better than Guenoche and Vitte's solutions.

From the above results, it is apparent that THSB outperformed Guenoche and Vitte's method, DPH, LR and EA for alphabets of size 4. Table 4 evaluates these heuristics on various alphabet sizes. In developing these instances, little information

Table 4 The performance of THSB and other heuristics on large k-LCSP instances with various alphabets

$\|\Sigma\|$	Num of Seq.	Length of Seq.	G and V		LR		EA		THSB with $q = 0.66$					
			Avg CS	Avg Time (s)	Avg CS	Avg Time (s)	Avg CS	Avg Time (s)	p value	Avg CS	Avg Time (s)	p value	Avg CS	Avg Time (s)
2	10	1000	562.82	0.02	477.3	0.01	505.4	2.0	20	562.0	0.4	40	577.24	24.52
	50	1000	503.72	0.06	463.4	0.03	483.9	141.4	20	506.1	1.0	40	511.3	85.08
	100	1000	489.56	0.14	459.4	0.06	477.8	1067.5	20	493.2	2.2	40	497.88	196.24
10	10	1000	153.38	0.02	96.2	0.01	113.4	3.9	30	156.7	2.0	50	162.5	90.64
	50	1000	105.38	0.06	86.9	0.04	88.0	362.8	30	107.6	0.8	50	109.78	69.6
	100	1000	96.62	0.16	83.7	0.06	N/A	N/A	30	98.7	1.1	50	100.7	58.6
25	10	2500	183.64	0.06	98.9	0.02	110.9	26.6	30	173.8	8.8	60	188.92	102.2
	50	2500	112.74	0.2	88.2	0.09	N/A	N/A	30	106.8	2.2	80	115.26	52.44
	100	2500	101.52	0.4	84.7	0.17	N/A	N/A	30	97.7	2.5	90	104.14	81.1
100	10	5000	113.6	0.2	51.4	0.08	75.7	105.2	30	92.2	36.0	150	117.78	6, 099.28
	50	5000	58.38	0.66	42.8	0.41	N/A	N/A	30	52.2	185.7	250	60.86	4, 272.984
	100	5000	50.44	1.28	40.1	0.82	N/A	N/A	30	46.5	353.7	300	52.66	11, 128.32
Average			211	0.28	172.7	0.12	N/A	N/A		207.8	49.7		216.6	1, 855.2

is gained by changing n (see Table 3 for its effect). Thus, a sufficiently large n is chosen for each alphabet and the value of k is changed for each instance. Again the instances in Table 4 are randomly created with each letter being chosen with a probability of $1/|\Sigma|$. Each row again represents the average of 50 random instances.

DPH performed so poorly in Table 3 that it is not included on Table 4. Furthermore, EA required more than 30 minutes to solve some instances, which is denoted by an N/A. Since SB requires $O(k|\Sigma|^{|L^{2^*}|})$ effort, p varies depending upon $|\Sigma|$ as shown in Table 4. Values of p are chosen so that one solution is generated fast and the other requires more time, similar to $p = 20$ and $p = 40$ for $|\Sigma| = 4$. For some alphabets as k increased, the value of p could also increase due to the shorter expected length of L^{2^*}.

Selecting appropriate p values was difficult when $|\Sigma| = 100$ because each SB instance requires $O(k(100)^{|L^{2^*}|})$ effort. No reasonable p value ran fast. Both $p = 20$ and $p = 30$ required over 30 seconds per problem with 10 sequences and 5,000 letters. A p value of 50 could be solved in a reasonable amount of time (less than a minute), but the reported common subsequences were inferior to Guenoche and Vitte's common sequences. To generate good common subsequences, large p values were required. These large p values required substantially more time than the other problems (see Table 4).

Table 4 demonstrates that THSB produces better common subsequences than the other three heuristics. THSB with the larger p values and $q = .66$ created solutions that are on average 2.7% better than G and V, 25.4% better than LR and 15.1% better than EA (on the instances that EA could solve).

The reader should be aware that THSB's solutions were substantially better than the forward solutions generated from G and V. However, after G and V's method ran backwards and the two solutions were merged, the gap between THSB's and G and V's solutions decreased by a large margin.

5 Conclusions and future research

This paper has defined large neighborhoods for k-LCSP and used optimal solutions of these neighborhoods to create a new heuristic, THSB, to solve k-LCSP. The computational study demonstrated that THSB provides better solutions than the existing heuristics, but may require more time. From this study, we conclude that if an individual does not need great k-LCSP solutions, then they should implement Guenoche and Vitte's heuristic. However, if the individual needs excellent k-LCSP solutions, then he/she should implement THSB and seek a computer that can solve instances with large p values.

This paper has also raised some important future research directions. The strength of Guenoche and Vitte's heuristic results from running their process both forward and backward, and then merging the two solutions. Since THSB only run forwards, incorporating these ideas into THSB would approximately double THSB's time and should improve THSB's solutions. In addition, THSB only incorporated p neighborhoods, so can other neighborhoods provide better solutions to k-LCSP instances? Finally, can the time horizon principle of sequentially stepping through a problem

and throwing away a portion of an optimal solution be applied to other combinatorial optimization problems?

Acknowledgements The authors would like to thank an anonymous referee for some valuable suggestions. The authors would also like to thank Dr. Alain Guenoche for some valuable comments and suggestions relating to his work on LCS. This research was partially supported by the Kansas Technology Enterprise Corporation.

References

Aho, A.V., Hopcroft, J.E., Ullman, J.: Data Structures and Algorithms. Addison–Wiley, Reading (1983)
Ahuja, R., Ergun, O., Orlin, J., Punen, A.: A survey of very large-scale neighborhood search techniques. Discret. Appl. Math. **123**(1–3), 75–102 (2002)
Bafna, V., Muthukrishnan, S., Ravi, R.: Computing similarity between RNA strings. In: Proceedings of the 6th Annual Symposium on Combinatorial Pattern Matching, pp. 1–16, 1995
Banerjee, A., Ghosh, J.: Clickstream clustering using weighted longest common subsequences. In: Proceedings of the Web Mining Workshop at the 1st SIAM Conference on Data Mining, pp. 33–40, 2001
Bergroth, L., Hakonen, H., Raitta, T.: New approximation algorithms for longest common subsequences. In: Proceedings. String Processing and Information Retrieval: A South American Symposium, pp. 32–40, 1998
Bergroth, L., Hakonen, H., Raitta, T.: A survey of longest common subsequence algorithms. In: Proceedings Seventh International Symposium on String Processing and Information Retrieval. SPIRE 2000, pp. 39–48, 2000
Bonizzoni, P., Vedova, G.D., Mauri, G.: Experimenting an approximation algorithm for the LCS. Discret. Appl. Math. **110**(1), 13–24 (1998)
Brisk, P., Kaplan, A., Sarrafzadeh, M.: Area-efficient instruction set synthesis for reconfigurable system-on-chip designs. In: Proceedings 2004 Design Automation Conference, pp. 395–400, 2004
Chin, F., Poon, C.: Performance analysis of some simple heuristics for computing longest common subsequences. Algorithmica **12**(4–5), 293–311 (1994)
Dayhoff, M.O.: Computer analysis of protein evolution. Sci. Am. **221**(1), 86–95 (1969)
Dayhoff, M., Schwartz, R., Orcutt, B.: A model of evolutionary change in proteins. Atlas Protein Seq. Struct. **5**, 345–352 (1978)
Eppstein, D., Galil, Z., Giancarlo, R.: Italiano, Sparse dynamic programming. II. convex and concave cost functions. J. Assoc. Comput. Mach. **39**(3), 546–567 (1992)
Gallant, J., Maier, D., Storer, J.A.: On finding minimal length superstrings. J. Comput. Syst. Sci. **20**(1), 50–58 (1980)
Guenoche, A., Vitte, P.: Longest common subsequence to multiple strings. Exact and approximate algorithms. Tech. Sci. Inform. **14**(7), 897–915 (1995)
Guenoche, A.: Supersequence of masks for oligo-chips. J. Bioinform. Comput. Biol. **2**(3), 459–469 (2004)
Hakata, K., Imai, H.: The longest common subsequence problem for small alphabet size between many strings. In: Proceedings of the 3rd International Symposium on Algorithms and Computation, 650, pp. 469–478, 1992
Hayes, C.C.: A model of planning for plan efficiency: taking advantage of operator overlap. In: Proceedings of the 11th International Joint Conference of Artificial Intelligence, pp. 949–953, 1989
Hirschberg, D.S.: A linear space algorithm for computing maximal common subsequences. Commun. Assoc. Comput. Mach. **18**(6), 341–343 (1975)
Hirschberg, D.S.: Algorithms for the longest common subsequence problem. J. Assoc. Comput. Mach. **24**(4), 664–675 (1977)
Hsu, W.J., Du, M.W.: Computing a longest common subsequence for a set of strings. BIT **24**, 45–59 (1984)
Hunt, J.W., McIlroy, M.D.: An algorithm for differential file comparison. Computing Science Technical Report, 41, AT&T Bell Laboratories, Murray Hill, New Jersey (1975)
Hunt, J.W., Szymanski, T.G.: A fast algorithm for computing longest common subsequences. Commun. Assoc. Comput. Mach. **20**(5), 350–353 (1977)
Irving, R.W., Fraser, C.B.: Two algorithms for the longest common subsequence of three (or more) strings. In: Proceedings of 3rd Symposium on Combinatorial Pattern Matching, vol. 644, pp. 214–229. Springer, Berlin (1992)

Itoga, S.Y.: The string merging problem. BIT **21**, 20–30 (1981)
Jiang, T., Li, M.: On the approximation of shortest common and longest common subsequences. SIAM J. Comput. **24**(5), 1122–1139 (1995)
Jiang, T., Lin, G., Ma, B., Zhang, K.: A general edit distance between RNA structures. J. Comput. Biol. **9**(2), 371–388 (2002)
Land, A.H., Doig, A.G.: An automatic method for solving discrete programming problems. Econometrica **28**, 497–520 (1960)
Larson, R.: State Increment Dynamic Programming. Elsevier, New York (1968)
Lu, S.Y., Fu, K.S.: A sentence-to-sentence clustering procedure for pattern analysis. IEEE Trans. Syst. Man Cybern. **SMC-8**(5), 381–389 (1978)
Maier, D.: The complexity of some problems on subsequences and supersequences. J. Assoc. Comput. Mach. **25**, 322–336 (1978)
Sankoff, D., Kruskal, J.B. (eds.): Time Warps, String Edits, and Macromolecules: the Theory and Practice of Sequence Comparison. Addison–Wesley, Reading (1983)
Sellis, T.: Multiple query optimization. ACM Trans. Database Syst. **13**(1), 23–52 (1988)
Singireddy, A.: Solving the longest common subsequence problem in Bioinformatics. Master's Thesis, Industrial and Manufacturing Systems Engineering, Kansas State University, Manhattan, KS (2003)
Smith, T.F., Waterman, M.S.: Identification of common molecular subsequences. J. Mol. Biol. **147**, 195–197 (1981)
Storer, J.: Data Compression: Methods and Theory. Computer Science Press, MD (1988)
Thompson, J.D., Higgins, D.G., Gibson, T.J.: CLUSTAL W: improving the sensitivity of progressive multiple sequence alignment through sequence weighting, position-specific gap penalties and weight matrix choice. Nucleic Acids Res. **22**, 4673–4680 (1994)
Wagner, R.A.: Common phrases and minimum-space text storage. Commun. Assoc. Comput. Mach. **16**(3), 148–152 (1973)
Winston, W.L.: Operations Research Applications and Algorithms, 4th edn. Brooks/Cole-Thomson Learning, Belmont (2004)